

Day 12: Constructors

- Default constructor
- Parameterized constructor
- Constructor overloading
- this keyword

First Understand One Thing

Previous day manam **Classes and Objects** nerchukunnam.

Example:

```
class Student
```

```
{  
    String name;  
    int age;  
}
```

Object create chestham.

```
Student s1 = new Student();
```

Question...

Object create ayindi...

But...

Name?

Age?

Nothing.

Java automatic ga object create chesindi.

Appudu question...

"Object create ayye time lone values assign cheyyali ante?"

Answer:

Constructor.

What is Constructor?

Simple definition:

Constructor ante object create ayye moment lo automatic ga execute ayye special method.

Remember this sentence.

Object create avvagane...

Constructor run avutundi.

Example

```
Student s1 = new Student();
```

Java internally ila chestundi.

Memory allocate

↓

Constructor call

↓

Object ready

Constructor ni manam call cheyyamu.

Java automatic ga call chestundi.

Why Constructor?

Without constructor

```
Student s1 = new Student();
```

```
s1.name="Ravi";
```

```
s1.age=20;
```

First object create.

Tarvata values assign.

Constructor unte

```
Student s1=new Student("Ravi",20);
```

One line lo

Object create

-

Values initialize

Done.

Real Life Example

Imagine...

Car showroom.

Car order chestaru.

Factory car build chestundi.

Engine install chestundi.

Seats install chestundi.

Paint chestundi.

Everything prepare chesi car istundi.

Meeru malli engine install cheyyaru.

Exactly same.

Constructor

↓

Object ni ready state lo prepare chestundi.

Constructor Syntax

```
class Student
```

```
{
```

```
    Student()
```

```
{
```

```
}  
}
```

Observe carefully.

Constructor name

↓

Student

Class name

↓

Student

Both same.

Rules of Constructor

Rule 1

Constructor name should be same as class name.

Correct

```
class Student
```

```
{  
    Student()  
    {  
  
    }  
}
```

Wrong

```
class Student
```

```
{  
    Demo()  
    {  
  
    }  
}
```

```
}
```

Rule 2

No return type.

Correct

```
Student()
```

```
{
```

```
}
```

Wrong

```
void Student()
```

```
{
```

```
}
```

Wrong

```
int Student()
```

```
{
```

```
}
```

Wrong

```
String Student()
```

```
{
```

```
}
```

Constructor ki return type undadu.

Rule 3

Automatically called.

No need

```
s1.Student();
```

Wrong.

Java itself calls.

Rule 4

Constructor can have parameters.

Example

```
Student(String n,int a)
```

```
{
```

```
}
```

Rule 5

Can be overloaded.

Same name

Different parameters.

Life Cycle of Object

Suppose

```
Student s1=new Student();
```

Java internally

Step 1

Memory allocate

Heap

name=null

age=0

Step 2

Constructor execute

Suppose constructor

Student()

```
{  
    name="Unknown";  
    age=18;  
}
```

Now memory

name="Unknown"

age=18

Step 3

Object ready.

Default Constructor

Most important topic.

Question.

Suppose

class Student

```
{
```

```
}
```

No constructor.

Still

Student s1=new Student();

Runs.

Why?

Because

Java automatically creates

Default Constructor.

Internally

Student()

{

}

Java itself adds.

This is called

Default Constructor.

Example

class Student

{

}

Equivalent

class Student

{

 Student()

 {

 }

}

Important Rule

Java creates default constructor only when

YOU DON'T WRITE ANY CONSTRUCTOR.

Suppose


```
class Student
{
    Student()
    {

    }
}
```

Java won't create another constructor.

Suppose

```
class Student
{
    Student(int x)
    {

    }
}
```

Now

```
Student s=new Student();
```

Error.

Why?

Java won't generate default constructor now.

Only constructor available

```
Student(int x)
```

Need

```
Student s=new Student(10);
```

Example 1

```
class Student
```

```
{  
    Student()  
    {  
        System.out.println("Constructor called");  
    }  
  
    public static void main(String args[])  
    {  
        Student s1=new Student();  
    }  
}
```

Output

Constructor called

Object create ayyaka immediate ga constructor execute ayindi.

Example 2

Student s1=new Student();

Student s2=new Student();

Student s3=new Student();

Output

Constructor called

Constructor called

Constructor called

Every object has its own constructor call.

Parameterized Constructor

Suppose

Every student different.

Need values.

Student()

{

}

Enough aa?

No.

Need

Name

Age

Marks

Example

class Student

{

String name;

int age;

Student(String n,int a)

{

name=n;

age=a;

}

}

Object

Student s1=new Student("Ravi",20);

Memory

name=Ravi

age=20

Second object

Student s2=new Student("Rahul",18);

Memory

name=Rahul

age=18

Every object different values.

Constructor Parameters vs Instance Variables

Student(String n,int a)

```
{  
    name=n;  
    age=a;  
}
```

Confusion.

n

↓

parameter

name

↓

instance variable

Parameter

Temporary.

Only constructor lopala.

Instance variable

Whole object life.

Visualization

Student("Ravi",20)

↓

n="Ravi"

a=20

↓

name=n

↓

name="Ravi"

Done.

Constructor Overloading

Already methods overloading nerchukunnam.

Same concept.

Multiple constructors.

Example

class Student

```
{  
    Student()  
    {  
  
    }  
  
    Student(String name)  
    {  
  
    }  
  
    Student(String name,int age)  
    {  
  
    }  
  
    Student(String name,int age,double fee)  
    {  
  
    }  
}
```

Same name.

Different parameters.

Valid.

Why Constructor Overloading?

Because

Every time same information undadu.

Case 1

```
Student s1=new Student();
```

No data.

Case 2

```
Student s2=new Student("Ravi");
```

Only name.

Case 3

```
Student s3=new Student("Ravi",20);
```

Name and age.

Case 4

```
Student s4=new Student("Ravi",20,55000);
```

Complete data.

Different situations.

Constructor Overloading Rules

Valid

```
Student()
```

```
Student(int x)
```

```
Student(String name)
```

```
Student(int x,String y)
```

Invalid

```
Student()
```

```
Student()
```

Same parameters.

Compiler error.

What is this Keyword?

Most confusing topic.

Don't worry.

Let's understand.

Suppose

```
class Student
```

```
{
```

```
    String name;
```

```
    Student(String name)
```

```
{
```

```
        name=name;
```

```
}
```

```
}
```

Question.

Which name?

Parameter?

Instance variable?

Compiler gets confused.

Actually

Both same name.

name

↓

parameter

name

↓

instance variable

Memory

Parameter

name="Ravi"

Instance variable

name=null

Statement

name=name;

Means

Parameter = Parameter

Instance variable unchanged.

Still

null

Solution

Use

this

class Student

{

String name;

Student(String name)

{

 this.name=name;

}

}

Now

this.name

↓

Current object's variable

name

↓

Parameter

Meaning

Object variable

=

Parameter value

Done.

What Exactly is "this"?

this ante **current object reference**.

Simple ga:

Object create ayina taruvata, aa object ni constructor/method lopala refer cheyyadaniki Java **this** ane keyword ni istundi.

Example:

```
Student s1 = new Student("Ravi");
```

Constructor execute avutunnappudu,

this

means

s1 object

If another object:

```
Student s2 = new Student("Priya");
```

Appudu

this

means

s2 object

So **this** always points to the object whose constructor or method is currently running.

Visualization

Object create

```
Student s1 = new Student("Ravi");
```

Memory

s1

|

▼

name = null

age = 0

Constructor starts

```
Student(String name)
```

```
{  
    this.name = name;  
}
```

At this moment

this

|



name = null

age = 0

Parameter

name = "Ravi"

Statement

this.name = name;

becomes

Object.name = "Ravi"

Final Memory

name = Ravi

age = 0

When do we use this?

1. Differentiate instance variable and parameter (Most common)

```
class Student
```

```
{  
    String name;
```

```
Student(String name)
{
    this.name = name;
}
}
```

2. Call another constructor in the same class

```
class Student
```

```
{
    String name;
    int age;

    Student()
    {
        this("Unknown",18);
    }
}
```

```
Student(String name,int age)
{
    this.name=name;
    this.age=age;
}
}
```

Flow:

```
new Student()
```

↓

Student()

↓

this("Unknown",18)

↓

Student(String,int)

↓

Object initialized

Rule: this(...) must be the **first statement** inside the constructor.

Correct:

Student()

```
{  
    this("Unknown",18);  
}
```

Wrong:

Student()

```
{  
    System.out.println("Hello");  
    this("Unknown",18); // Error  
}
```

3. Refer current object's methods

class Student

```
{
```

```
void show()
{
    System.out.println("Hello");
}
```

```
void display()
{
    this.show();
}
}
```

this.show();

Same as

show();

But using this makes it explicit that the current object's method is being called.

4. Refer current object's variables

this.name

this.age

this.marks

Constructor vs Method

Constructor	Method
Same name as class	Any valid name
No return type	Can have return type or void
Runs automatically during object creation	Called manually

Constructor	Method
Used to initialize object	Used to perform operations
Cannot return a value	Can return values

Constructor Chaining

Suppose

class Student

```
{
    Student()
    {
        System.out.println("Default");
    }

    Student(int x)
    {
        this();
        System.out.println("Parameterized");
    }
}
```

Object

Student s=new Student(10);

Output

Default

Parameterized

Flow

Student(int)

↓

this()

↓

Student()

↓

Back

↓

Student(int)

Can Constructor be Private?

Yes.

```
private Student()
```

```
{
```

```
}
```

Why?

Singleton Design Pattern.

Also used to prevent object creation from outside the class.

Can Constructor be Static?

No.

Wrong

```
static Student()  
{  
  
}
```

Compiler error.

Reason:

Constructor belongs to **object creation**, while static belongs to the **class**.
Constructor runs when an object is created, so it cannot be static.

Can Constructor be Final?

No.

Can Constructor be Abstract?

No.

Can Constructor be Inherited?

No.

Every class has its own constructors.

Methods can be inherited.

Constructors cannot.

Interview Questions

Q1. Constructor ki return type untunda?

No.

Q2. Constructor ni manual ga call cheyyacha?

No. Java automatically calls it during object creation.

Q3. Constructor overloading possible aa?

Yes.

Q4. Default constructor eppudu create avutundi?

Only when **no constructor is written** in the class.

Q5. this ante enti?

Current object reference.

Q6. this.name = name; enduku use chestham?

Parameter and instance variable names same unte, current object's variable ni refer cheyyadaniki.

Q7. Constructor return value istunda?

No.

Q8. Constructor static aa?

No.

Q9. Constructor private undacha?

Yes.

Q10. Constructor inheritance avutunda?

No.

Complete Real-Life Example

```
class Student
{
    String name;
    int age;
```

```
Student()
```

```
{  
    this("Unknown",18);  
}
```

```
Student(String name,int age)
```

```
{  
    this.name=name;  
    this.age=age;  
}
```

```
void display()
```

```
{  
    System.out.println("Name : " + name);  
    System.out.println("Age : " + age);  
}
```

```
public static void main(String[] args)
```

```
{  
    Student s1 = new Student();  
    Student s2 = new Student("Ravi",20);  
  
    s1.display();  
  
    System.out.println();  
  
    s2.display();  
}
```

}

Output

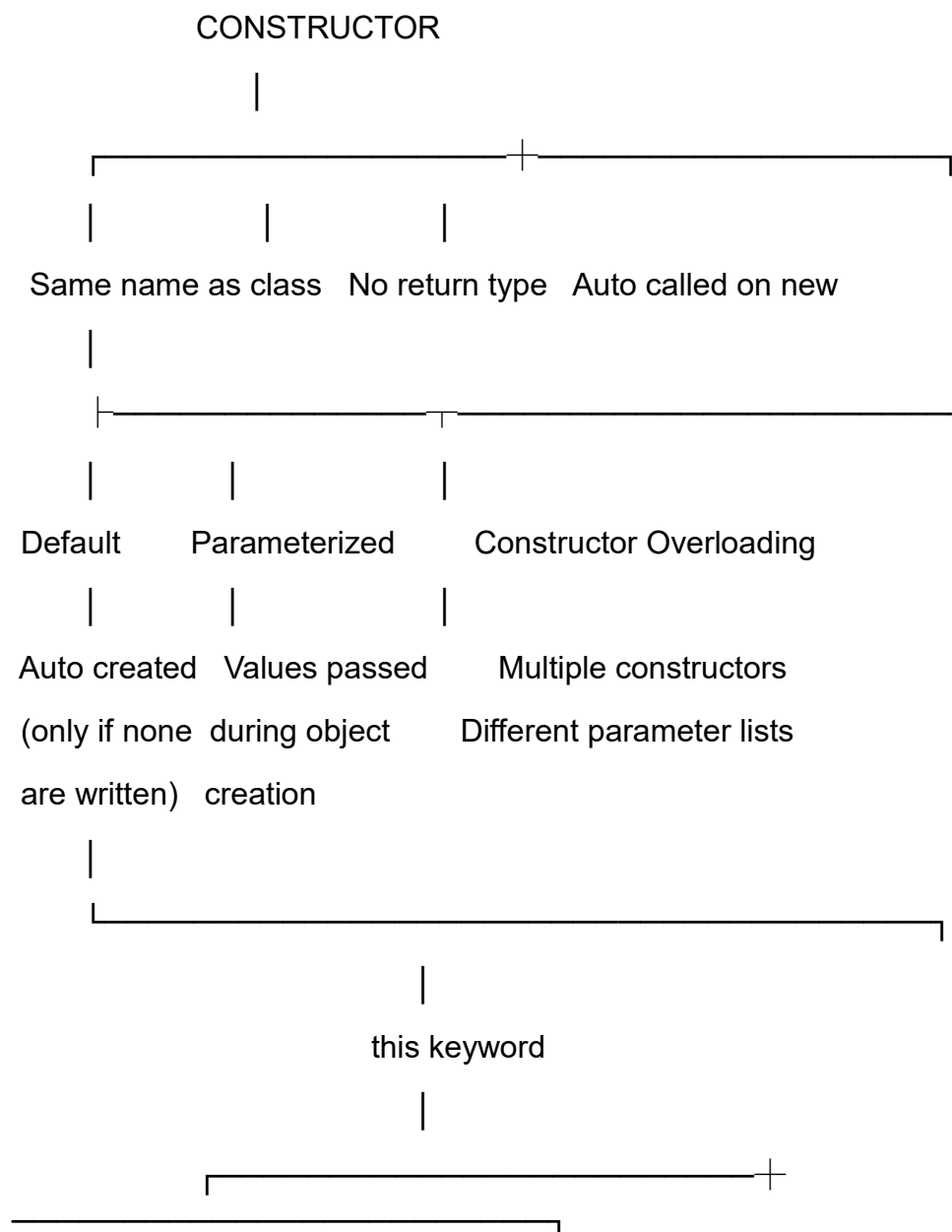
Name : Unknown

Age : 18

Name : Ravi

Age : 20

Mind Map (Remember Forever)



Current object reference	Access object fields	Call another constructor
(this)	(this.name)	(this(...))

Practice Programs

1. Create a Student class with a default constructor that prints "Student object created".
2. Create an Employee class with a parameterized constructor to initialize id, name, and salary.
3. Create a Book class with three overloaded constructors:
 - No parameters
 - Only title
 - Title and price
4. Create a Car class where constructor parameters have the same names as instance variables and use this to initialize them.
5. Create a Rectangle class with constructor chaining:
 - Default constructor should call a parameterized constructor using this(10, 20).

This keyword

"When the parameter and instance variable have the same name, this is required to distinguish them. But this can also be used in many other situations because it always refers to the current object."